

CITIZEN COMPLAINT MANAGEMENT SYSTEM (CCMS)

Submitted By

Udayraj Patil

Roll Number: **24BCE11159**

Course

B.Tech Computer Science and Engineering (CSE-Core)

Institution

VIT Bhopal University

Academic Year

2024-2028

1. INTRODUCTION

The Citizen Complaint Management System (CCMS) is a backend application developed using Java 25, Spring Boot 3.5.x, and MongoDB 8.3. The system provides a centralized platform for citizens to register complaints related to public services such as water supply, road maintenance, electricity, garbage collection, and street lighting.

The application enables government departments and officers to manage complaints, assign responsibilities, update complaint status, and maintain complaint records efficiently. REST APIs are tested using Postman and all data is stored in MongoDB.

2. PROJECT OBJECTIVE

The objective of this project is to bridge the gap between citizens and government departments by digitizing the grievance redressal process.

The system allows citizens to register complaints and track their status while enabling departments and officers to efficiently manage, assign, and resolve complaints.

The project implements:

- CRUD Operations
 - RESTful APIs
 - MongoDB Database Integration
 - Relationship Mapping
 - Complaint Status Tracking
 - Dashboard Analytics and Reporting
-

3. PROBLEM STATEMENT

Traditional complaint management systems rely heavily on manual records and disconnected workflows, resulting in delays and lack of transparency.

Major challenges include:

- Difficulty tracking complaint status
- Lack of centralized complaint records
- Inefficient complaint assignment
- Limited administrative reporting
- Absence of complaint history tracking

The Citizen Complaint Management System addresses these issues through a centralized, database-driven platform that supports complaint registration, assignment, monitoring, and resolution.

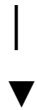
4. TECHNOLOGY STACK

Component	Technology
Programming	Java 25
Framework	Spring Boot 3.5.x

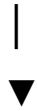
Component	Technology
Database	MongoDB 8.3
IDE	IntelliJ IDEA 2026.1.2
Build Tool	Maven 11.x
API Testing	Postman 3.9x
Database	MongoDB Compass
Server Port	8081

5. SYSTEM ARCHITECTURE

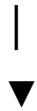
Postman Client



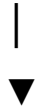
Controller Layer



Service Layer



Repository Layer



MongoDB Database

Layer Responsibilities

Controller Layer

- Handles HTTP Requests
- Maps REST Endpoints

- Returns Responses

Service Layer

- Contains Business Logic
- Performs Validations
- Handles Complaint Processing

Repository Layer

- Communicates with MongoDB
- Performs CRUD Operations

MongoDB

- Stores and manages all application data
-

6. DATABASE DESIGN

Citizen Collection

```
{  
  "_id": "String",  
  "name": "String",  
  "email": "String",  
  "mobile": "String"  
}
```

Department Collection

```
{  
  "_id": "String",  
  "departmentName": "String"  
}
```

Officer Collection

```
{  
  "_id": "String",
```

```
"officerName": "String",  
"departmentId": "String"  
}
```

Complaint Collection

```
{  
  "_id": "String",  
  "title": "String",  
  "description": "String",  
  "status": "PENDING | ASSIGNED | RESOLVED",  
  "citizenId": "String",  
  "departmentId": "String",  
  "officerId": "String"  
}
```

Complaint History Collection

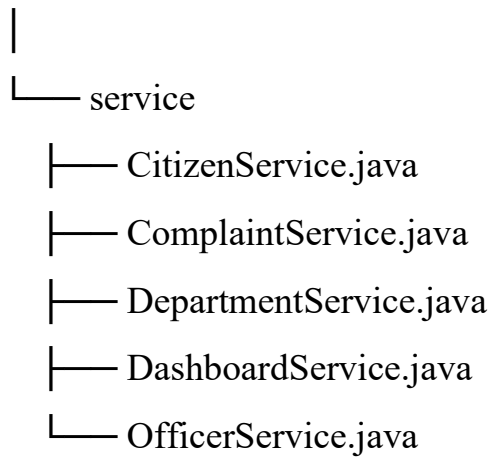
```
{  
  "_id": "String",  
  "complaintId": "String",  
  "status": "String",  
  "updatedAt": "LocalDateTime"  
}
```

Project structure

src/main/java

```
└── com  
    └── cms  
        ├── CitizenComplaintApplication.java  
        |
```

```
|— config
|
|— controller
|   |— AuthController.java
|   |— CitizenController.java
|   |— ComplaintController.java
|   |— DepartmentController.java
|   |— DashboardController.java
|   |— OfficerController.java
|
|— dto
|
|— exception
|
|— model
|   |— Citizen.java
|   |— Complaint.java
|   |— ComplaintHistory.java
|   |— Department.java
|   |— Officer.java
|
|— repository
|   |— CitizenRepository.java
|   |— ComplaintHistoryRepository.java
|   |— ComplaintRepository.java
|   |— DepartmentRepository.java
|   |— OfficerRepository.java
```



7. RELATIONSHIP MAPPING

One Citizen → Many Complaints

Citizen (1) → Complaint (M)

One Department → Many Complaints

Department (1) → Complaint (M)

One Department → Many Officers

Department (1) → Officer (M)

One Complaint → Many Status Updates

Complaint (1) → ComplaintHistory (M)

8. REST API MODULES

Authentication APIs

- POST /api/auth/register
- POST /api/auth/login

Citizen APIs

- POST /api/citizens
- GET /api/citizens
- GET /api/citizens/{id}

- PUT /api/citizens/{id}
- DELETE /api/citizens/{id}

Department APIs

- POST /api/departments
- GET /api/departments
- PUT /api/departments/{id}
- DELETE /api/departments/{id}

Officer APIs

- POST /api/officers
- GET /api/officers
- PUT /api/officers/{id}
- DELETE /api/officers/{id}

Complaint APIs

- POST /api/complaints
- GET /api/complaints
- GET /api/complaints/{id}
- PUT /api/complaints/{id}
- DELETE /api/complaints/{id}
- PUT /api/complaints/{id}/assign/{officerId}
- PUT /api/complaints/{id}/status/{status}
- GET /api/complaints/status/{status}
- GET /api/complaints/citizen/{citizenId}

Dashboard APIs

- GET /api/dashboard/total-complaints
- GET /api/dashboard/pending-complaints
- GET /api/dashboard/assigned-complaints
- GET /api/dashboard/resolved-complaints

- GET /api/dashboard/department-wise-report

9. SCREENSHOT SECTION

Screenshot 1 – Admin Registration

The screenshot displays a REST client interface for a POST request to `http://localhost:8081/api/auth/register`. The request body is a JSON object: `{ "username": "admin", "password": "password123", "role": "ADMIN" }`. The response body is also JSON: `{ "message": "User 'admin' registered successfully with role: ADMIN", "status": "Success" }`.

```
HTTP http://localhost:8081/api/auth/register

POST http://localhost:8081/api/auth/register

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 { "username": "admin", "password": "password123", "role": "ADMIN" }

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize

1 {
2   "message": "User 'admin' registered successfully with role: ADMIN",
3   "status": "Success"
4 }
```

Screenshot 2 – Admin Login

The screenshot displays a REST client interface for a POST request to `http://localhost:8081/api/auth/login`. The request body is a JSON object: `{ "username": "admin", "password": "password123" }`. The response body is a JSON object containing a message, status, and token: `{ "message": "User 'admin' logged in successfully.", "status": "Success", "token": "dummy-jwt-token-for-future-enhancement" }`.

```
HTTP http://localhost:8081/api/auth/login

POST http://localhost:8081/api/auth/login

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 { "username": "admin", "password": "password123" }

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize

1 {
2   "message": "User 'admin' logged in successfully.",
3   "status": "Success",
4   "token": "dummy-jwt-token-for-future-enhancement"
5 }
```

Screenshot 3 – Create Department

The screenshot shows a REST client interface with the URL `http://localhost:8081/api/departments`. The method is set to **POST**. The request body is in **raw** format and contains a JSON object:

```
{
  "departmentName": "Public Parks and Sanitation"
}
```

The response section shows the **Body** tab with the **JSON** format selected. The response body is a JSON object:

```
{
  "departmentId": "6a2ee325002da6b42b742eff",
  "departmentName": "Public Parks and Sanitation"
}
```

Screenshot 4 – View All Departments

The screenshot shows a REST client interface with the URL `http://localhost:8081/api/departments`. The method is set to **GET**. The response section shows the **Body** tab with the **JSON** format selected. The response body is a JSON array of five department objects:

```
[
  {
    "departmentId": "6a2ee2f1002da6b42b742efb",
    "departmentName": "Waste Management"
  },
  {
    "departmentId": "6a2ee2f8002da6b42b742efc",
    "departmentName": "Electricity Department"
  },
  {
    "departmentId": "6a2ee302002da6b42b742efd",
    "departmentName": "Water Supply Board"
  },
  {
    "departmentId": "6a2ee310002da6b42b742efe",
    "departmentName": "Roads and Transport"
  },
  {
    "departmentId": "6a2ee325002da6b42b742eff",
    "departmentName": "Public Parks and Sanitation"
  }
]
```

Screenshot 5 – Find Department By ID

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8081/api/departments/6a2ee302002da6b42b742efd`
- Method:** `GET`
- Body:** The response body is empty, with a line number `1` at the top.
- Headers:** 6 headers are present.
- Body Type:** `raw` is selected.
- JSON:** The response is in JSON format.
- Body Content:** The response body contains the following JSON object:

```
{
  "departmentId": "6a2ee302002da6b42b742efd",
  "departmentName": "Water Supply Board"
}
```

Screenshot 6 – Update Department

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8081/api/departments/6a2ee302002da6b42b742efd`
- Method:** `DELETE`
- Body:** The response body is empty, with a line number `1` at the top.
- Headers:** 5 headers are present.
- Body Type:** `raw` is selected.
- Raw:** The response is in raw format.
- Raw Content:** The response body contains the following text:

```
Department deleted successfully.
```

Screenshot 7 – Delete Department

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8081/api/departments/6a2ee302002da6b42b742efd`
- Method:** `DELETE`
- Body:** The body tab is selected, showing a single line: `1`.
- Format:** The format dropdown is set to `JSON`.
- Response:** The response tab is selected, showing a single line: `1 Department deleted successfully.`

Screenshot 8 – Create Officer

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8081/api/officers`
- Method:** `POST`
- Body:** The body tab is selected, showing a JSON object:

```
1 {  
2   "officerName": "Inspector Ravi",  
3   "departmentId": "6a2ee2f1002da6b42b742efb"  
4 }
```
- Format:** The format dropdown is set to `JSON`.
- Response:** The response tab is selected, showing a JSON object:

```
1 {  
2   "officerId": "6a2ee512002da6b42b742f01",  
3   "officerName": "Inspector Ravi",  
4   "departmentId": "6a2ee2f1002da6b42b742efb"  
5 }
```

Screenshot 9 – View All Officers

HTTP **http://localhost:8081/api/officers**

GET **http://localhost:8081/api/officers**

Docs Params Authorization Headers 8 **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

1 {
2
3 "officerId": "6a2ee512002da6b42b742f01",
4 "officerName": "Inspector Ravi",
5 "departmentId": "6a2ee2f1002da6b42b742efb"
6 },
7 {
8 "officerId": "6a2ee541002da6b42b742f02",
9 "officerName": "Sub-Inspector Anjali",
10 "departmentId": "Electricity Department"
11 },
12 {
13 "officerId": "6a2ee55d002da6b42b742f03",
14 "officerName": "Chief Officer Patel",
15 "departmentId": "6a2ee31002da6b42b742efe"
16 },
17 {
18 "officerId": "6a2ee5b6002da6b42b742f04",
19 "officerName": "Inspector Santosh Jadhav",
20 "departmentId": "6a2ee325002da6b42b742eff"
21 },
22 {
23 "officerId": "6a2ee5cd002da6b42b742f05",
24 "officerName": "Sub-Inspector Vaishali Kadam",
25 "departmentId": "6a2ee4f8002da6b42b742f00"
26 }
27]

Body Cookies Headers 5 Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

Screenshot 10 – Find Officer By ID

HTTP **http://localhost:8081/api/officers/6a2ee5cd002da6b42b742f05**

GET **http://localhost:8081/api/officers/6a2ee5cd002da6b42b742f05**

Docs **Params** Authorization Headers 6 **Body** Scripts Tests Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers 5 Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {  
2   "officerId": "6a2ee5cd002da6b42b742f05",  
3   "officerName": "Sub-Inspector Vaishali Kadam",  
4   "departmentId": "6a2ee4f8002da6b42b742f00"  
5 }
```

Screenshot 11 – Update Officer

The screenshot shows an HTTP client interface with the following details:

- URL:** `http://localhost:8081/api/officers/01225000`
- Method:** `DELETE`
- Body:** `{ "officerName": "Senior Inspector Ravi", "departmentId": "6a2ee2f1002da6b42b742efb" }`
- Response:** `1 Officer deleted successfully.`

Screenshot 12 – Delete Officer

The screenshot shows an HTTP client interface with the following details:

- URL:** `http://localhost:8081/api/officers/01225000`
- Method:** `PUT`
- Body:** `{ "officerName": "Senior Inspector Ravi", "departmentId": "6a2ee2f1002da6b42b742efb" }`
- Response:** `1 Officer deleted successfully.`

Screenshot 13 – Create Citizen

The screenshot shows an HTTP client interface with the following details:

- URL:** `http://localhost:8081/api/citizens`
- Method:** `POST`
- Body:** `{ "name": "Pooja Desai", "email": "pooja@example.com", "mobile": "9876543210" }`
- Response:** `{ "citizenId": "6a2ee727002da6b42b742f06", "name": "Pooja Desai", "email": "pooja@example.com", "mobile": "9876543210" }`

Screenshot 14 – View All Citizens

http://localhost:8081/api/citizens

GET

http://localhost:8081/api/citizens

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

1

Body

Cookies

Headers 5

Test Results

200 OK • 15 ms • 1.06 KB • ...

JSON

Preview

Visualize

	citizenId	name	email	mobile
0	6a2e336ccbaad8f7486e33ee	Rahul Sharma	rahul.sharma@example.com	9876543210
1	6a2e361748991395f13224fa	Amit Patel	amit.patel@example.com	9876543211
2	6a2e363148991395f13224fb	Priya Sharma	priya.sh@example.com	9876543212
3	6a2e365048991395f13224fc	Rohan Desai	rohan.d@example.com	9999999999
4	6a2e366a48991395f13224fd	Sneha Reddy	sneha.r@example.com	9876543214
5	6a2e367548991395f13224fe	Vikram Singh	vikram.s@example.com	9876543215
6	6a2e367f48991395f13224ff	Ananya Gupta	ananya.g@example.com	9876543216
7	6a2ee727002da6b42b742f06	Pooja Desai	pooja@example.com	9876543210

Screenshot 15 – Find Citizen By ID

HTTP

http://localhost:8081/api/citizens/6a2e366a48991395f13224fd

GET

http://localhost:8081/api/citizens/6a2e366a48991395f13224fd

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

Query Params

Key	Value
Key	Value

Body

Cookies

Headers 5

Test Results

JSON

Preview

Visualize

```
1 {
2   "citizenId": "6a2e366a48991395f13224fd",
3   "name": "Sneha Reddy",
4   "email": "sneha.r@example.com",
5   "mobile": "9876543214"
6 }
```

Screenshot 16 – Delete Citizen

HTTP **DELETE** `http://localhost:8081/api/citizens/0123456789`

Docs Params Authorization Headers 6 **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

1

Body Cookies Headers 5 Test Results ↻

Raw ▾ ▶ Preview 🖼 Visualize ▾

1 Citizen deleted successfully.

Screenshot 17 – Create Complaint

HTTP **POST** `http://localhost:8081/api/complaints`

switch request type **POST** ▾ `http://localhost:8081/api/complaints`

Docs Params Authorization Headers 8 **Body** ● Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "title": "Low Water Pressure",
3   "description": "We are getting water for only 10 minutes a day with incredibly low pressure.",
4   "citizenId": "6a2e367f48991395f13224ff",
5   "departmentId": "6a2ee4f8002da6b42b742f00"
6 }
```

Body Cookies Headers 5 Test Results ↻ **200 OK** ⚙

JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "complaintId": "6a3149781bd1d2a231fe4699",
3   "title": "Low Water Pressure",
4   "description": "We are getting water for only 10 minutes a day with incredibly low pressure.",
5   "status": "PENDING",
6   "citizenId": "6a2e367f48991395f13224ff",
7   "departmentId": "6a2ee4f8002da6b42b742f00",
8   "officerId": null
9 }
```


Screenshot 18 – Assign Complaint To Officer

HTTP

http://localhost:8081/api/complaints/6a3149781bd1d2a231fe4699/assign/6a2ee5b6002da6b42b742f04

PUT

http://localhost:8081/api/complaints/6a3149781bd1d2a231fe4699/assign/6a2ee5b6002da6b42b742f04

Docs

Params

Authorization

Headers 7

Body

Scripts

Tests

Settings

Query Params

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers 5

Test Results

{ } JSON

Preview

Visualize

1

{

2

"complaintId": "6a3149781bd1d2a231fe4699",

3

"title": "Low Water Pressure",

4

"description": "We are getting water for only 10 minutes a day with incredibly low pressure.",

5

"status": "ASSIGNED",

6

"citizenId": "6a2e367f48991395f13224ff",

7

"departmentId": "6a2ee4f8002da6b42b742f00",

8

"officerId": "6a2ee5b6002da6b42b742f04"

9

}

Screenshot 19 – Update Complaint Status

HTTP

http://localhost:8081/api/complaints/status/RESOLVED

GET

http://localhost:8081/api/complaints/status/RESOLVED

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

Query Params

Key	Value	Descr
Key	Value	Descr

Body

Cookies

Headers 5

Test Results

{ } JSON

Preview

Visualize

1

[

2

{

3

"complaintId": "6a2e33d8cbaad8f7486e33ef",

4

"title": "No Water Supply",

5

"description": "We have not received water supply for the last two days in Sector 4.",

6

"status": "RESOLVED",

7

"citizenId": "6a2e336ccbaad8f7486e33ee",

8

"departmentId": "WATER_DEPT_01",

9

"officerId": "6a2ee512002da6b42b742f01"

10

}

11

]

Screenshot 20 – Find Complaints By Citizen

http://localhost:8081/api/complaints/citizen/6a2e336ccbaad8f7486e33ee

GEThttp://localhost:8081/api/complaints/citizen/6a2e336ccbaad8f7486e33ee

DocsParamsAuthorizationHeaders6BodyScriptsTestsSettings

Query Params

Key	Value	Description
Key	Value	Description

BodyCookiesHeaders5Test Results🕒

{ } JSONPreview🖨️ Visualize

```
1  [
2      {
3          "complaintId": "6a2e33d8cbaad8f7486e33ef",
4          "title": "No Water Supply",
5          "description": "We have not received water supply for the last two days in Sector 4.",
6          "status": "RESOLVED",
7          "citizenId": "6a2e336ccbaad8f7486e33ee",
8          "departmentId": "WATER_DEPT_01",
9          "officerId": "6a2ee512002da6b42b742f01"
10     }
11 ]
```

Screenshot 21 – View All Complaints

http://localhost:8081/api/complaints

GEThttp://localhost:8081/api/complaints

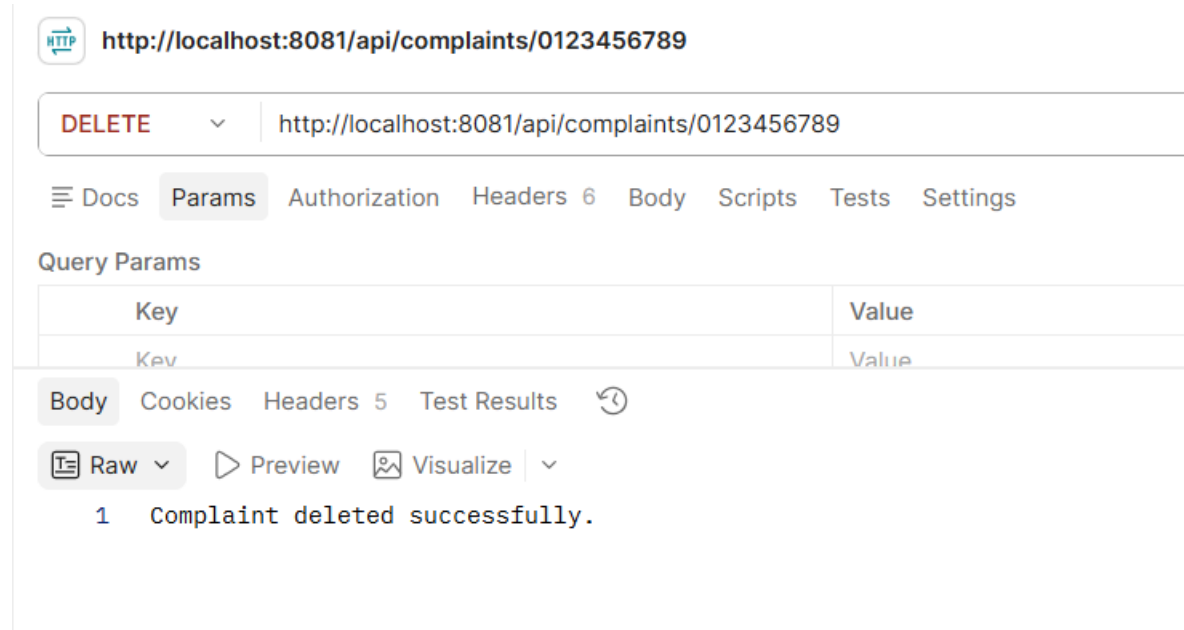
DocsParamsAuthorizationHeaders8BodyScriptsTestsSettings

BodyCookiesHeaders5Test Results🕒200 OK87 ms2.1 KB🌐...

{ } JSONPreview🖨️ Visualize

	complaintId	title	description	status	citizenId	departmentId	officerId
0	6a2e33d8cbaad8f7486e33ef	No Water Supply	We have not received water supply for the last two days in Sector 4.	RESOLVED	6a2e336ccbaad8f7486e33ee	WATER_DEPT_01	6a2ee512002da6b42b742f01
1	6a2ee9da002da6b42b742f09	Major Drinking Water Pipeline Leak	There is a massive leak in the main drinking water pipeline near the central market. Thousands of liters are being wasted.	ASSIGNED	6a2e361748991395f13224fa	6a2ee4f8002da6b42b742f00	6a2ee541002da6b42b742f02
2	6a2eea09002da6b42b742f0b	Continuous Power Fluctuations	We are experiencing severe voltage fluctuations for the past 48 hours, which has damaged several household appliances.	PENDING	6a2e363148991395f13224fb	6a2ee2f8002da6b42b742efc	null
3	6a2eea3d002da6b42b742f0d	Dangerous Potholes on Highway Link	Multiple deep potholes have developed on the highway link road after the recent rain, causing two minor accidents already.	PENDING	6a2e365048991395f13224fc	6a2ee310002da6b42b742efe	null
4	6a2eea69002da6b42b742f0f	Broken Playground Equipment	The children's swings and slides in Shivaji Park are broken and rusted, posing a serious safety risk to kids.	RESOLVED	6a2e366a48991395f13224fd	6a2ee325002da6b42b742eff	6a2ee5cd002da6b42b742f05
5	6a3149781bd1d2a231fe4699	Low Water Pressure	We are getting water for only 10 minutes a day with incredibly low pressure.	ASSIGNED	6a2e367f48991395f13224ff	6a2ee4f8002da6b42b742f00	6a2ee5b6002da6b42b742f04

Screenshot 22 – Delete Complaint



HTTP **DELETE** `http://localhost:8081/api/complaints/0123456789`

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Query Params

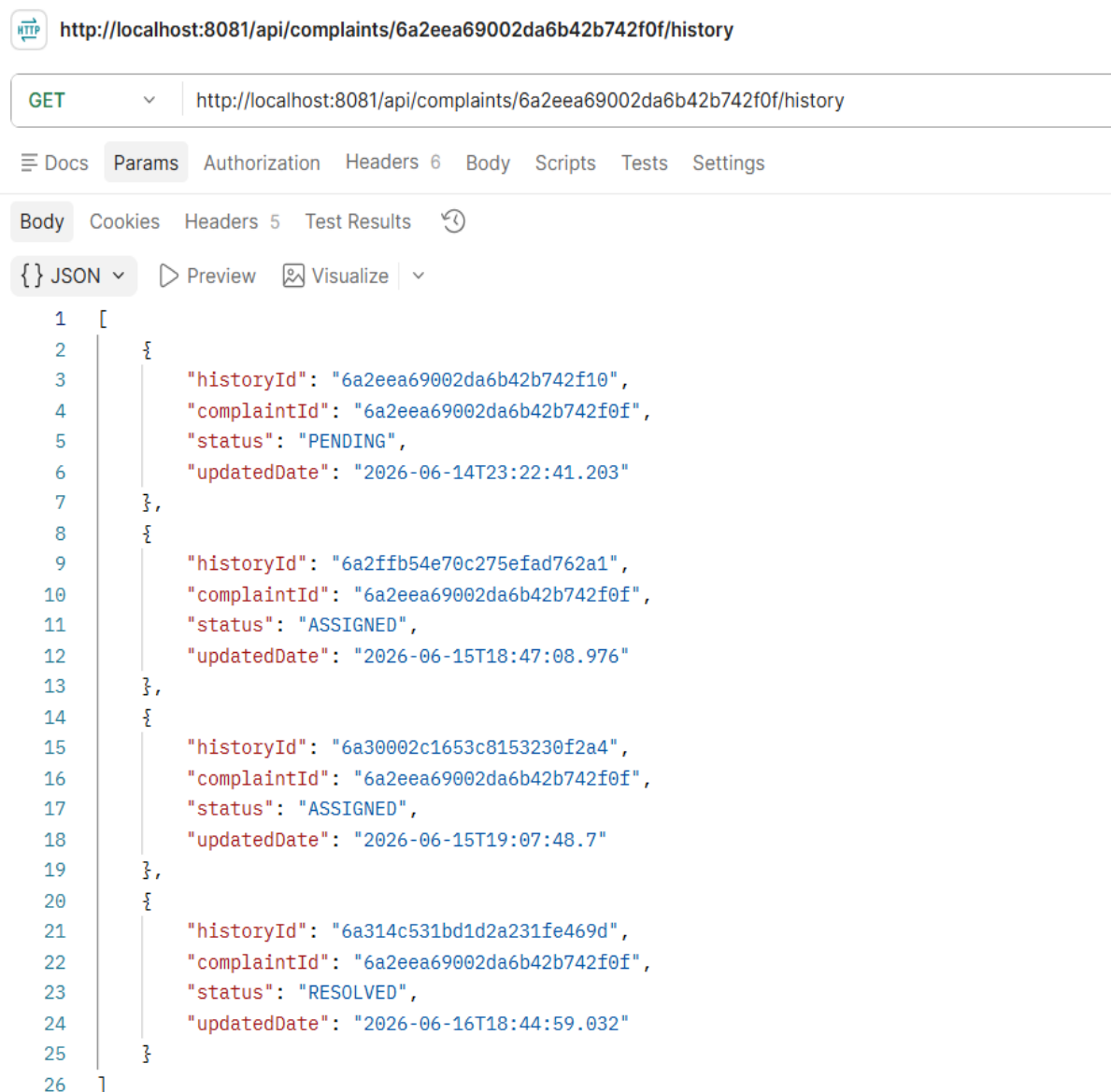
Key	Value
Key	Value

Body Cookies Headers 5 Test Results ↺

Raw Preview Visualize

```
1 Complaint deleted successfully.
```

Screenshot 23 – Complaint Lifecycle History



HTTP **GET** `http://localhost:8081/api/complaints/6a2eea69002da6b42b742f0f/history`

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 5 Test Results ↺

{ } JSON Preview Visualize

```
1 [
2   {
3     "historyId": "6a2eea69002da6b42b742f10",
4     "complaintId": "6a2eea69002da6b42b742f0f",
5     "status": "PENDING",
6     "updatedAt": "2026-06-14T23:22:41.203"
7   },
8   {
9     "historyId": "6a2ffb54e70c275efad762a1",
10    "complaintId": "6a2eea69002da6b42b742f0f",
11    "status": "ASSIGNED",
12    "updatedAt": "2026-06-15T18:47:08.976"
13  },
14  {
15    "historyId": "6a30002c1653c8153230f2a4",
16    "complaintId": "6a2eea69002da6b42b742f0f",
17    "status": "ASSIGNED",
18    "updatedAt": "2026-06-15T19:07:48.7"
19  },
20  {
21    "historyId": "6a314c531bd1d2a231fe469d",
22    "complaintId": "6a2eea69002da6b42b742f0f",
23    "status": "RESOLVED",
24    "updatedAt": "2026-06-16T18:44:59.032"
25  }
26 ]
```

Screenshot 24 – Dashboard Analytics: Total Complaints

HTTP `http://localhost:8081/api/dashboard/total-complaints`

GET `http://localhost:8081/api/dashboard/total-complaints`

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 5 Test Results

Raw Preview Visualize

1 Total Complaints Registered: 6

Screenshot 25 – Dashboard Analytics: Pending Complaints

HTTP `http://localhost:8081/api/dashboard/pending-complaints`

GET `http://localhost:8081/api/dashboard/pending-complaints`

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 5 Test Results

Raw Preview Visualize

1 Pending Complaints: 2

Screenshot 26 – Dashboard Analytics: Assigned Complaints

HTTP `http://localhost:8081/api/dashboard/assigned-complaints`

GET `http://localhost:8081/api/dashboard/assigned-complaints`

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 5 Test Results

Raw Preview Visualize

1 Assigned Complaints: 2

Screenshot 27 – Dashboard Analytics: Resolved Complaints

HTTP

http://localhost:8081/api/dashboard/resolved-complaints

GET

http://localhost:8081/api/dashboard/resolved-complaints

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

Body

Cookies

Headers 5

Test Results

Raw

Preview

Visualize

1

Resolved Complaints: 2

Screenshot 28 – Department Wise Complaint Report

HTTP

http://localhost:8081/api/dashboard/department-wise-report

GET

http://localhost:8081/api/dashboard/department-wise-report

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

Body

Cookies

Headers 5

Test Results

JSON

Preview

Visualize

1

{

2

"6a2ee310002da6b42b742efe": 1,

3

"6a2ee2f8002da6b42b742efc": 1,

4

"6a2ee4f8002da6b42b742f00": 2,

5

"WATER_DEPT_01": 1,

6

"6a2ee325002da6b42b742eff": 1

7

}

Screenshot 29 – MongoDB Citizen Collection

localhost:27017 > citizen_complaint_db > Citizen

Open MongoDB shell

Documents 8

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

25

1 - 8 of 8

Refresh

Previous

Next

Filter

Sort

Grid

Citizen

	_id ObjectId	name String	email String	mobile String	_class String	
1	ObjectId('6a2e336ccbada8f...	"Rahul Sharma"	"rahul.sharma@example.com"	"9876543210"	"com.cms.model.Citizen"	✎ 📄 🗑
2	ObjectId('6a2e36174899139...	"Amit Patel"	"amit.patel@example.com"	"9876543211"	"com.cms.model.Citizen"	✎ 📄 🗑
3	ObjectId('6a2e36314899139...	"Priya Sharma"	"priya.sh@example.com"	"9876543212"	"com.cms.model.Citizen"	✎ 📄 🗑
4	ObjectId('6a2e36504899139...	"Rohan Desai"	"rohan.d@example.com"	"9999999999"	"com.cms.model.Citizen"	✎ 📄 🗑
5	ObjectId('6a2e366a4899139...	"Sneha Reddy"	"sneha.r@example.com"	"9876543214"	"com.cms.model.Citizen"	✎ 📄 🗑
6	ObjectId('6a2e36754899139...	"Vikram Singh"	"vikram.s@example.com"	"9876543215"	"com.cms.model.Citizen"	✎ 📄 🗑
7	ObjectId('6a2e367f4899139...	"Ananya Gupta"	"ananya.g@example.com"	"9876543216"	"com.cms.model.Citizen"	✎ 📄 🗑
8	ObjectId('6a2ee727002da6b...	"Pooja Desai"	"pooja.desai@example.com"	"9999988888"	"com.cms.model.Citizen"	✎ 📄 🗑

Screenshot 30 – MongoDB Complaint Collection

localhost:27017 > citizen_complaint_db > Complaint

Documents6

Aggregations

Schema

Indexes1

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

251 - 6 of 6

	_id ObjectId	title String	description String	status String	citizenId String	de
1	ObjectId('6a2e33d8cbaad8f...	"No Water Supply"	"We have not received wat...	"RESOLVED"	"6a2e336ccbaad8f7486e33ee"	"W
2	ObjectId('6a2ee9da002da6b...	"Major Drinking Water Pip...	"There is a massive leak ...	"ASSIGNED"	"6a2e361748991395f13224fa"	"6
3	ObjectId('6a2eea9002da6b...	"Continuous Power Fluctua...	"We are experiencing seve...	"PENDING"	"6a2e363148991395f13224fc"	"6
4	ObjectId('6a2eea3d002da6b...	"Dangerous Potholes on Hi...	"Multiple deep potholes h...	"PENDING"	"6a2e365048991395f13224fd"	"6
5	ObjectId('6a2eea69002da6b...	"Broken Playground Equipm...	"The children's swings an...	"RESOLVED"	"6a2e366a48991395f13224fe"	"6
6	ObjectId('6a3149781bd1d2a...	"Low Water Pressure"	"We are getting water for...	"ASSIGNED"	"6a2e367f48991395f13224ff"	"6

Screenshot 31 – MongoDB Complaint History Collection

localhost:27017 > citizen_complaint_db > ComplaintHistory

Documents16

Aggregations

Schema

Indexes1

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

251 - 16 of 16

	_id ObjectId	complaintId String	status String	updatedAt Date	_class String	
1	ObjectId('6a2e33d8cbaad8f...	"6a2e33d8cbaad8f7486e33ef"	"PENDING"	2026-06-14T04:53:44.326+0...	"com.cms.model.Complainth...	
2	ObjectId('6a2e348acbaad8f...	"6a2e33d8cbaad8f7486e33ef"	"RESOLVED"	2026-06-14T04:56:42.944+0...	"com.cms.model.Complainth...	
3	ObjectId('6a2ee99b002da6b...	"6a2ee99b002da6b42b742f07"	"PENDING"	2026-06-14T17:49:15.404+0...	"com.cms.model.Complainth...	
4	ObjectId('6a2ee9da002da6b...	"6a2ee9da002da6b42b742f09"	"PENDING"	2026-06-14T17:50:18.067+0...	"com.cms.model.Complainth...	
5	ObjectId('6a2eea9002da6b...	"6a2eea9002da6b42b742f0b"	"PENDING"	2026-06-14T17:51:05.218+0...	"com.cms.model.Complainth...	
6	ObjectId('6a2eea3d002da6b...	"6a2eea3d002da6b42b742f0d"	"PENDING"	2026-06-14T17:51:57.550+0...	"com.cms.model.Complainth...	
7	ObjectId('6a2eea69002da6b...	"6a2eea69002da6b42b742f0f"	"PENDING"	2026-06-14T17:52:41.203+0...	"com.cms.model.Complainth...	
8	ObjectId('6a2ff9d0586d4cd...	"6a2e33d8cbaad8f7486e33ef"	"ASSIGNED"	2026-06-15T13:10:40.589+0...	"com.cms.model.Complainth...	
9	ObjectId('6a2ffa39586d4cd...	"6a2ee9da002da6b42b742f09"	"ASSIGNED"	2026-06-15T13:12:25.158+0...	"com.cms.model.Complainth...	
10	ObjectId('6a2ffb54e70c275...	"6a2eea69002da6b42b742f0f"	"ASSIGNED"	2026-06-15T13:17:08.976+0...	"com.cms.model.Complainth...	
11	ObjectId('6a300061653c81...	"6a2e33d8cbaad8f7486e33ef"	"RESOLVED"	2026-06-15T13:37:10.466+0...	"com.cms.model.Complainth...	
12	ObjectId('6a30002c1653c81...	"6a2eea69002da6b42b742f0f"	"ASSIGNED"	2026-06-15T13:37:48.700+0...	"com.cms.model.Complainth...	
13	ObjectId('6a3149781bd1d2a...	"6a3149781bd1d2a231fe4699"	"PENDING"	2026-06-16T13:02:48.710+0...	"com.cms.model.Complainth...	
14	ObjectId('6a314aba1bd1d2a...	"6a3149781bd1d2a231fe4699"	"ASSIGNED"	2026-06-16T13:08:10.644+0...	"com.cms.model.Complainth...	
15	ObjectId('6a314af51bd1d2a...	"6a3149781bd1d2a231fe4699"	"ASSIGNED"	2026-06-16T13:09:09.413+0...	"com.cms.model.Complainth...	
16	ObjectId('6a314c531bd1d2a...	"6a2eea69002da6b42b742f0f"	"RESOLVED"	2026-06-16T13:14:59.032+0...	"com.cms.model.Complainth...	

Screenshot 32 – MongoDB Department Collection

localhost:27017 > citizen_complaint_db > Department

Documents5

Aggregations

Schema

Indexes1

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

251 - 5 of 5

	_id ObjectId	departmentName String	_class String	
1	ObjectId('6a2ee2f1002da6b...	"Waste Management"	"com.cms.model.Department"	
2	ObjectId('6a2ee2f8002da6b...	"Electricity Department"	"com.cms.model.Department"	
3	ObjectId('6a2ee310002da6b...	"Roads and Transport"	"com.cms.model.Department"	
4	ObjectId('6a2ee325002da6b...	"Public Parks and Sanitat...	"com.cms.model.Department"	
5	ObjectId('6a2ee4f8002da6b...	"Water Supply"	"com.cms.model.Department"	

Screenshot 33 – MongoDB Officer Collection

localhost:27017 > citizen_complaint_db > Officer

Documents 5 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA BULK EXPORT DATA EXPORT CODE

25 1-5 of 5

Officer

	_id ObjectId	officerName String	departmentId String	_class String	
1	ObjectId('6a2ee512002da6b...	"Inspector Ravi"	"6a2ee2f1002da6b42b742efb"	"com.cms.model.Officer"	
2	ObjectId('6a2ee541002da6b...	"Sub-Inspector Anjali"	"Electricity Department"	"com.cms.model.Officer"	
3	ObjectId('6a2ee55d002da6b...	"Chief Officer Patel"	"6a2ee310002da6b42b742efe"	"com.cms.model.Officer"	
4	ObjectId('6a2ee5b6002da6b...	"Inspector Santosh Jadhav"	"6a2ee325002da6b42b742eff"	"com.cms.model.Officer"	
5	ObjectId('6a2ee5cd002da6b...	"Sub-Inspector Vaishali Kadam"	"6a2ee4f8002da6b42b742f00"	"com.cms.model.Officer"	

10. TESTING

All APIs were tested using Postman.

Testing includes:

- Create Operations
- Read Operations
- Update Operations
- Delete Operations
- Complaint Assignment
- Complaint Tracking
- Dashboard Reporting

11. FUTURE ENHANCEMENTS

- JWT Authentication
- Email Notifications
- SMS Notifications
- File Upload Support

- Admin Dashboard
 - MongoDB Replication
 - MongoDB Sharding
-

12. CONCLUSION

The Citizen Complaint Management System provides an efficient and centralized platform for handling public grievances and improving communication between citizens and government departments. The project successfully demonstrates the implementation of CRUD operations, RESTful API development, MongoDB integration, relationship mapping, complaint assignment, complaint history tracking, and dashboard analytics using Java 25, Spring Boot 3.5.x, and MongoDB 8.3.

The system enables citizens to register and track complaints while allowing departments and officers to manage, assign, and resolve issues effectively. The inclusion of complaint lifecycle tracking and dashboard reporting enhances transparency, accountability, and overall operational efficiency.

Through this project, practical knowledge of backend development, Spring Boot architecture, MongoDB document databases, REST API design, and Postman testing was gained. The system serves as a scalable foundation for future enhancements such as authentication, notifications, file uploads, and advanced reporting features, making it a valuable solution for modern digital grievance management.